

Comprehensive Application Security Code Review Guide

This guide shows a complete security code review process, from selecting an application and language, to use. SAST tools and manual inspection, documenting findings, and providing remediation recommendations.

Below I've structured the response to workflow while incorporating security best practices from OWASP and other industry standards.

Python Web Application

I have used a Python Flask web application as our audit target. Python is widely used, has well-documented security pitfalls, and benefits from excellent SAST tooling.

Sample Vulnerable Code Snippet (test.py)

```
from flask import Flask, request, render_template_string, session, redirect import sqlite3 import hashlib

app = Flask(name) app.secret_key = 'hardcoded-secret-key-123' # VULNERABILITY: Hardcoded secret

@app.route('/login', methods=['POST']) def login(): username = request.form.get('username') password = request.form.get('password')

# VULNERABILITY: SQL Injection via string concatenation
query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
conn = sqlite3.connect('database.db')
cursor = conn.execute(query)
user = cursor.fetchone()

if user:
    session['user'] = username
    return redirect('/dashboard')
return "Login failed"

@app.route('/user/') def greet(name): # VULNERABILITY: Reflected XSS - no output encoding return render_template_string(f"Hello, {name}!")

if name == 'main': app.run(debug=True) # VULNERABILITY: Debug mode enabled in production
```

Why Python/Flask as an example?

- Common enterprise framework with real-world deployment
- Demonstrates multiple vulnerability classes (injection, XSS, crypto, configuration)
- Extensive SAST tooling available

Secure Coding Recommendations

1. Input Validation & Injection Prevention

Example of python code that can be vulnerable to the system

```
query = f"SELECT * FROM users WHERE username = '{username}'"  
cursor.execute(query)
```

In order to prevent from attacks, code can be secured through the following.

```
import re  
def validate_username(username):  
    # Allow-list pattern for username format if not  
    re.match(r'^[a-zA-Z0-9_-]{3,20}$', username):  
        raise ValueError('Invalid username format')  
    return username
```

2. Output Encoding & XSS Prevention

Example of a vulnerable XSS pattern

```
return render_template_string(f"Hello, {name}!")
```

To prevent from XSS we can secure through autoescaping templates.

```
templates/greet.html
```

```
Hello, {{ name }}! # Jinja autoescapes by default  
return render_template('greet.html', name=name)
```

3. Authentication & Secret Management

Example of vulnerable pattern, hardcoded secret.

```
app.secret_key = 'hardcoded-secret-key-123'
```

We can prevent through enabling environment variable and secret manager

```
import os  
from secrets import token_hex
```

Development fallback, production must set env var

```
app.secret_key = os.environ.get('FLASK_SECRET_KEY', token_hex(32))
```

Cryptography Best Practices

- **Never** invent custom encryption algorithms
- **Never** use encoding (Base64) as encryption

- **Always** use secure random numbers (`secrets` module, not `random`)
- **Never** log passwords, private keys, or tokens in plain text

Configuration & Error Handling

Example of vulnerable Pattern (Debug in Production)

```
app.run(debug=True)
```

This can be prevented through secure configuration.

```
import os
debug_mode = os.environ.get('FLASK_ENV') == 'development'
app.run(debug=debug_mode)
```